

Related topics

- [Launch the Windows Settings app](#)
-

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Handle the Windows camera privacy setting

Article • 06/24/2024

Windows allows users to grant or deny access to the device's camera in the Windows Settings app, under **Privacy & Security** -> **Camera**. Camera access can be disabled for the entire device, for all unpackaged apps, or for individual packaged apps. This article describes the best practices for checking whether your app has access to the camera and handling the case where access is denied by the user.

Check for access before initializing the camera

For packaged apps, you should check to see if your app has camera access before initializing the camera. Use the [AppCapability](#) class to determine if your app has access.

C#

C#

```
bool cameraCapabilityAccess = false;
private void CheckCameraAccessStatus()
{
    var status = AppCapability.Create("Webcam").CheckAccess();

    if (status == AppCapabilityAccessStatus.Allowed)
    {
        cameraCapabilityAccess = true;
        cameraButton.IsEnabled = true;
    }
    else
    {
        cameraCapabilityAccess = false;
        cameraButton.IsEnabled = false;
    }
}
```

Handle the access denied error

The Windows camera capture APIs will return the error **E_ACCESSDENIED** when apps attempt to access the camera capture device if the user has disabled camera in the camera privacy Settings page. Apps should check for this error when initializing the

capture device. If the initialization fails with this error, it is recommended that you direct the user to the camera privacy Settings page and potentially enable access for your app. The camera privacy Settings page can be launched using the URI `ms-settings:privacy-webcam`.

The following example illustrates how to check for `E_ACCESSDENIED` when calling [MediaCapture.InitializeAsync](#).

C#

C#

```
try
{
    await
mediaCapture.InitializeAsync(mediaCaptureInitializationSettings);
}
catch (System.UnauthorizedAccessException ex)
{
    // E_ACCESSDENIED, 0x80070005 in hexadecimal, -2147024891 in decimal
    if (ex.HResult == -2147024891)
    {
        StatusTextBlock.Text = "Access to the camera has been denied." +
            "Click the Settings button to check the camera privacy
settings";
    }

    return;
}
...
// Launch the camera privacy Settings page
private async void LaunchSettingsButton_Click(object sender,
RoutedEventArgs e)
{
    bool result = await Windows.System.Launcher.LaunchUriAsync(new
Uri("ms-settings:privacy-webcam"));
}
```

The following example illustrates handling the `E_ACCESSDENIED` error returned from [IMFActivate::ActivateObject](#) when initializing an [IMFMediaSource](#) for a capture device.

C++

```
IMFMediaSource* pSource = NULL;
IMFAttributes* pAttributes = NULL;
IMFActivate** ppDevices = NULL;

// Create an attribute store to specify the enumeration parameters.
```

```

HRESULT hr = MFCreateAttributes(&pAttributes, 1);
if (FAILED(hr))
{
    goto done;
}

// Source type: video capture devices
hr = pAttributes->SetGUID(
    MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE,
    MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE_VIDCAP_GUID
);
if (FAILED(hr))
{
    goto done;
}

// Enumerate devices.
UINT32 count;
hr = MFEnumDeviceSources(pAttributes, &ppDevices, &count);
if (FAILED(hr))
{
    goto done;
}

if (count == 0)
{
    hr = E_FAIL;
    goto done;
}

// Create the media source object.
hr = ppDevices[0]->ActivateObject(IID_PPV_ARGS(&pSource));
if (FAILED(hr))
{
    if (hr == E_ACCESSDENIED)
    {
        int response = MessageBox(hWnd, L"Access to the camera was denied.
Open the camera privacy settings?", L"Error", MB_YESNO);
        if (response == IDYES)
        {
            ShellExecute(NULL, L"open", L"ms-settings:privacy-webcam", L"",
L".", SW_SHOWDEFAULT);
        }
    }
    goto done;
}
}

```

Implement fallback behavior

Apps should implement the previous steps to alert the user detect and alert the user that camera access is restricted due to privacy settings and to direct the user to the

camera privacy Settings page to allow them to update their settings. After these steps, the app should retry camera initialization to see if access has been granted. If the user declines to update their settings to allow your app to access to the camera, consider providing alternative functionality. For example, you could disable camera features, switch to a different mode, or display a placeholder image in place of the camera preview.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Media playback

Article • 10/20/2022

This section provides information on creating Universal Windows apps that playback audio and video.

Media playback developer features

The following table lists the how-to articles that provide detailed guidance for adding media playback features to your app.

Topic	Description
Play audio and video with MediaPlayer	This article shows you how to take advantage of the new features and improvements to the media playback system for UWP apps. Starting with Windows 10, version 1607, the recommended best practice for playing media is to use the MediaPlayer class instead of MediaElement for media playback. The lightweight XAML control, MediaPlayerElement , has been introduced to allow you render media content in a XAML page. MediaPlayer provides several advantages including automatic integration with the System Media Transport Controls and a simpler, one-process model for background audio. This article also shows you how to render video to a Windows.UI.Composition surface and how to use a MediaTimelineController to synchronize multiple media players.
Media items, playlists, and tracks	This article shows you how to use the MediaSource class, which provides a common way to reference and play back media from different sources such as local or remote files and exposes a common model for accessing media data, regardless of the underlying media format. The MediaPlaybackItem class extends the functionality of MediaSource , allowing you to manage and select from multiple audio, video, and metadata tracks contained in a media item. MediaPlaybackList allows you to create playback lists from one or more media playback items.
Integrate with the System Media Transport Controls	This article shows you how to integrate your app with the System Media Transport Controls (SMTC). Starting with Windows 10, version 1607, every instance of MediaPlayer that you create to play media is automatically displayed by the SMTC. This article shows you how to provide the SMTC with metadata about the content you are playing and how to augment or completely override the default behavior of SMTC controls.
System-supported timed metadata cues	This article describes how to take advantage of several formats of timed metadata that may be embedded in media files or streams.

Topic	Description
Create, schedule, and manage media breaks	This article shows you how to create, schedule, and manage media breaks to your media playback app. Starting with Windows 10, version 1607, you can use the MediaBreakManager class to quickly and easy add media breaks to any MediaPlaybackItem that you play with a MediaPlayer . Media breaks are typically used to insert audio or video ads into media content. Once you schedule one or more media breaks, the system will automatically play your media content at the specified time during playback. The MediaBreakManager provides events so that your app can react when media breaks start, end, or when they are skipped by the user. You can also access a MediaPlaybackSession for your media breaks to monitor events like download and buffering progress updates.
Play media in the background	This article shows you how to configure your app so that media continues to play when your app moves from the foreground to the background. This means that even after the user has minimized your app, returned to the home screen, or has navigated away from your app in some other way, your app can continue to play audio. With Windows 10, version 1607, a new single-process model for background media playback has been introduced that is much quicker and easier to implement than the legacy two-process model. This article includes information on handling the new application lifecycle events EnteredBackground and LeavingBackground to manage your app's memory usage while running in the background.
Adaptive Streaming	This article describes how to add playback of adaptive streaming multimedia content to a Universal Windows Platform (UWP) apps. This feature currently supports playback of Http Live Streaming (HLS) and Dynamic Streaming over HTTP (DASH) content.
Media casting	This article shows you how to cast media to remote devices from a Universal Windows app.
Enable audio playback from remote Bluetooth-connected devices	This article shows you how to use AudioPlaybackConnection to enable Bluetooth-connected remote devices to play back audio on the local machine, enabling scenarios such as configuring a PC to behave like a Bluetooth speaker and allowing users to hear audio from their phone.
PlayReady DRM	This topic describes how to add PlayReady protected media content to your Universal Windows Platform (UWP) app.
PlayReady Encrypted Media Extension	This section describes how to modify your PlayReady Web app to support the changes made from the previous Windows 8.1 version to the Windows 10 version.

Media playback SDK samples

The following SDK samples demonstrate the media playback features available to UWP apps on Windows 10. Use these samples to see the media playback APIs used in context or as a starting point for your own app.

- [Adaptive streaming sample](#) 
- [Background Audio sample](#) 
- [System Media Transport sample](#) 

Play audio and video with MediaPlayer

Article • 04/16/2024

This article shows you how to play media in your Universal Windows app using the [MediaPlayer](#) class. With Windows 10, version 1607, significant improvements were made to the media playback APIs, including a simplified single-process design for background audio, automatic integration with the System Media Transport Controls (SMTTC), the ability to synchronize multiple media players, the ability to render video frames to a Windows.UI.Composition surface, and an easy interface for creating and scheduling media breaks in your content. To take advantage of these improvements, the recommended best practice for playing media is to use the **MediaPlayer** class instead of **MediaElement** for media playback. The lightweight XAML control, [MediaPlayerElement](#), has been introduced to allow you render media content in a XAML page. Many of the playback control and status APIs provided by **MediaElement** are now available through the new [MediaPlaybackSession](#) object. **MediaElement** continues to function to support backwards compatibility, but no additional features will be added to this class.

This article will walk you through the **MediaPlayer** features that a typical media playback app will use. Note that **MediaPlayer** uses the [MediaSource](#) class as a container for all media items. This class allows you to load and play media from many different sources, including local files, memory streams, and network sources, all using the same interface. There are also higher-level classes that work with **MediaSource**, such as [MediaPlaybackItem](#) and [MediaPlaybackList](#), that provide more advanced features like playlists and the ability to manage media sources with multiple audio, video, and metadata tracks. For more information on **MediaSource** and related APIs, see [Media items, playlists, and tracks](#).

ⓘ Note

Windows 10 N and Windows 10 KN editions do not include the media features required to use **MediaPlayer** for playback. These features can be installed manually. For more information, see [Media feature pack for Windows 10 N and Windows 10 KN editions](#) [↗](#).

Play a media file with MediaPlayer

Basic media playback with **MediaPlayer** is very simple to implement. First, create a new instance of the **MediaPlayer** class. Your app can have multiple **MediaPlayer** instances active at once. Next, set the [Source](#) property of the player to an object that implements

the **IMediaPlaybackSource**, such as a **MediaSource**, a **MediaPlaybackItem**, or a **MediaPlaybackList**. In this example, a **MediaSource** is created from a file in the app's local storage, and then a **MediaPlaybackItem** is created from the source and then assigned to the player's **Source** property.

Unlike **MediaElement**, **MediaPlayer** does not automatically begin playback by default. You can start playback by calling **Play**, by setting the **AutoPlay** property to true, or waiting for the user to initiate playback with the built-in media controls.

C#

```
mediaPlayer = new MediaPlayer();  
mediaPlayer.Source = MediaSource.CreateFromUri(new Uri("ms-appx:///Assets/example_video.mkv"));  
mediaPlayer.Play();
```

When your app is done using a **MediaPlayer**, you should call the **Close** method (projected to **Dispose** in C#) to clean up the resources used by the player.

C#

```
mediaPlayer.Dispose();
```

Use MediaPlayerElement to render video in XAML

You can play media in a **MediaPlayer** without displaying it in XAML, but many media playback apps will want to render the media in a XAML page. To do this, use the lightweight **MediaPlayerElement** control. Like **MediaElement**, **MediaPlayerElement** allows you to specify whether the built-in transport controls should be shown.

XML

```
<MediaPlayerElement x:Name="_mediaPlayerElement"  
AreTransportControlsEnabled="False" HorizontalAlignment="Stretch"  
Grid.Row="0"/>
```

You can set the **MediaPlayer** instance that the element is bound to by calling **SetMediaPlayer**.

C#

```
_mediaPlayerElement.SetMediaPlayer(mediaPlayer);
```

You can also set the playback source on the **MediaPlayerElement** and the element will automatically create a new **MediaPlayer** instance that you can access using the **MediaPlayer** property.

⚠ Note

Setting **MediaPlayerElement** properties will set the corresponding properties on its underlying **MediaPlayer**. You have the option to use the underlying **MediaPlayer** directly instead of using **MediaPlayerElement** properties. Be aware that using **MediaPlayer** directly where an equivalent **MediaPlayerElement** property could otherwise be used can cause unexpected behavior. This is because the **MediaPlayerElement** is not aware of everything happening to its underlying **MediaPlayer**. For example, if you set the source directly on **MediaPlayer**, then **MediaPlayerElement** **Source** property will not reflect the change. For this reason, you must be consistent in using **MediaPlayerElement** properties or directly using the underlying **MediaPlayer**.

C#

```
_mediaPlayerElement.Source = MediaSource.CreateFromUri(new Uri("ms-appx:///Assets/example_video.mkv"));
mediaPlayer = _mediaPlayerElement.MediaPlayer;
mediaPlayer.Play();
```

⚠ Note

If you disable the **MediaPlaybackCommandManager** of the **MediaPlayer** by setting **IsEnabled** to false, it will break the link between the **MediaPlayer** the **TransportControls** provided by the **MediaPlayerElement**, so the built-in transport controls will no longer automatically control the playback of the player. Instead, you must implement your own controls to control the **MediaPlayer**.

The **MediaPlayer** is detached from **MediaPlayerElement** when the **MediaPlayerElement** is destroyed or when a new **MediaPlayer** is set using **SetMediaPlayer**. When detached, **MediaPlayerElement** treats the underlying **MediaPlayer** differently depending on whether it was created by **MediaPlayerElement** or set using **SetMediaPlayer**.

If the **MediaPlayer** was created by **MediaPlayerElement**, it will properly [Close](#) the **MediaPlayer** for you. If the **MediaPlayer** was set on **MediaPlayerElement** using **SetMediaPlayer**, you are responsible for ensuring the **MediaPlayer** is properly closed. Failing to do so may result in fatal playback errors in **MediaPlayer**. The following code snippet shows how to properly detach and close in code.

C#

```
// Get a reference to the current media source.
IMediaPlaybackSource _mediaPlayerElement = _mediaPlayerElement.Source;

// Pause playback if able.
if (mediaPlayer.PlaybackSession.CanPause)
{
    mediaPlayer.Pause();
}

// Disconnect the MediaPlayer from its source. This can be done by setting
// the MediaPlayerElement Source property to null or by directly setting the
// source to null on the underlying MediaPlayer.
_mediaPlayerElement.Source = null;

// Disconnect the MediaPlayer from MediaPlayerElement.
_mediaPlayerElement.SetMediaPlayer(null);

// Dispose of the MediaPlayer or Source if they're no longer needed.
if (source is MediaSource mediaSource)
{
    mediaSource.Dispose();
}

mediaPlayer.Dispose();
```

Common MediaPlayer tasks

This section shows you how to use some of the features of the **MediaPlayer**.

Set the audio category

Set the [AudioCategory](#) property of a **MediaPlayer** to one of the values of the [MediaPlayerAudioCategory](#) enumeration to let the system know what kind of media you are playing. Games should categorize their music streams as **GameMedia** so that game music mutes automatically if another application plays music in the background. Music or video applications should categorize their streams as **Media** or **Movie** so they will take priority over **GameMedia** streams.

C#

```
mediaPlayer.AudioCategory = MediaPlayerAudioCategory.Media;
```

Output to a specific audio endpoint

By default, the audio output from a **MediaPlayer** is routed to the default audio endpoint for the system, but you can specify a specific audio endpoint that the **MediaPlayer** should use for output. In the example below, [MediaDevice.GetAudioRenderSelector](#) returns a string that uniquely identifies the audio render category of devices. Next, the [DeviceInformation](#) method [FindAllAsync](#) is called to get a list of all available devices of the selected type. You may programmatically determine which device you want to use or add the returned devices to a [ComboBox](#) to allow the user to select a device.

C#

```
string audioSelector = MediaDevice.GetAudioRenderSelector();
var outputDevices = await DeviceInformation.FindAllAsync(audioSelector);
foreach (var device in outputDevices)
{
    var deviceItem = new ComboBoxItem();
    deviceItem.Content = device.Name;
    deviceItem.Tag = device;
    _audioDeviceComboBox.Items.Add(deviceItem);
}
```

In the [SelectionChanged](#) event for the devices combo box, the [AudioDevice](#) property of the **MediaPlayer** is set to the selected device, which was stored in the [Tag](#) property of the **ComboBoxItem**.

C#

```
private void _audioDeviceComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    DeviceInformation selectedDevice = (DeviceInformation)
((ComboBoxItem)_audioDeviceComboBox.SelectedItem).Tag;
    if (selectedDevice != null)
    {
        mediaPlayer.AudioDevice = selectedDevice;
    }
}
```

Playback session

As described previously in this article, many of the functions that are exposed by the **MediaElement** class have been moved to the **MediaPlaybackSession** class. This includes information about the playback state of the player, such as the current playback position, whether the player is paused or playing, and the current playback speed. **MediaPlaybackSession** also provides several events to notify you when the state changes, including the current buffering and download status of content being played and the natural size and aspect ratio of the currently playing video content.

The following example shows you how to implement a button click handler that skips 10 seconds forward in the content. First, the **MediaPlaybackSession** object for the player is retrieved with the **PlaybackSession** property. Next the **Position** property is set to the current playback position plus 10 seconds.

C#

```
private void _skipForwardButton_Click(object sender, RoutedEventArgs e)
{
    var session = mediaPlayer.PlaybackSession;
    session.Position = session.Position + TimeSpan.FromSeconds(10);
}
```

The next example illustrates using a toggle button to toggle between normal playback speed and 2X speed by setting the **PlaybackRate** property of the session.

C#

```
private void _speedToggleButton_Checked(object sender, RoutedEventArgs e)
{
    mediaPlayer.PlaybackSession.PlaybackRate = 2.0;
}
private void _speedToggleButton_Unchecked(object sender, RoutedEventArgs e)
{
    mediaPlayer.PlaybackSession.PlaybackRate = 1.0;
}
```

Starting with Windows 10, version 1803, you can set the rotation with which video is presented in the **MediaPlayer** in increments of 90 degrees.

C#

```
mediaPlayer.PlaybackSession.PlaybackRotation =
    MediaRotation.Clockwise90Degrees;
```

Detect expected and unexpected buffering

The **MediaPlaybackSession** object described in the previous section provides two events for detecting when the currently playing media file begins and ends buffering, **BufferingStarted** and **BufferingEnded**. This allows you to update your UI to show the user that buffering is occurring. Initial buffering is expected when a media file is first opened or when the user switches to a new item in a playlist. Unexpected buffering can occur when the network speed degrades or if the content management system providing the content experiences technical issues. Starting with RS3, you can use the **BufferingStarted** event to determine if the buffering event is expected or if it is unexpected and interrupting playback. You can use this information as telemetry data for your app or media delivery service.

Register handlers for the **BufferingStarted** and **BufferingEnded** events to receive buffering state notifications.

C#

```
mediaPlayer.PlaybackSession.BufferingStarted +=  
MediaPlaybackSession_BufferingStarted;  
mediaPlayer.PlaybackSession.BufferingEnded +=  
MediaPlaybackSession_BufferingEnded;
```

In the **BufferingStarted** event handler, cast the event args passed into the event to a **MediaPlaybackSessionBufferingStartedEventArgs** object and check the **IsPlaybackInterruption** property. If this value is true, the buffering that triggered the event is unexpected and interrupting playback. Otherwise, it is expected initial buffering.

C#

```
private void MediaPlayerSession_BufferingStarted(MediaPlaybackSession  
sender, object args)  
{  
    MediaPlayerSessionBufferingStartedEventArgs bufferingStartedEventArgs  
= args as MediaPlayerSessionBufferingStartedEventArgs;  
    if (bufferingStartedEventArgs != null &&  
bufferingStartedEventArgs.IsPlaybackInterruption)  
    {  
        // update the playback quality telemetry report to indicate that  
        // playback was interrupted  
    }  
  
    // update the UI to indicate that playback is buffering  
}  
private void MediaPlayerSession_BufferingEnded(MediaPlaybackSession  
sender, object args)  
{  
    // update the UI to indicate that playback is no longer buffering  
}
```

Pinch and zoom video

MediaPlayer allows you to specify the source rectangle within video content that should be rendered, effectively allowing you to zoom into video. The rectangle you specify is relative to a normalized rectangle (0,0,1,1) where 0,0 is the upper left hand of the frame and 1,1 specifies the full width and height of the frame. So, for example, to set the zoom rectangle so that the top-right quadrant of the video is rendered, you would specify the rectangle (.5,0,.5,.5). It is important that you check your values to make sure that your source rectangle is within the (0,0,1,1) normalized rectangle. Attempting to set a value outside of this range will cause an exception to be thrown.

To implement pinch and zoom using multi-touch gestures, you must first specify which gestures you want to support. In this example, scale and translate gestures are requested. The **ManipulationDelta** event is raised when one of the subscribed gestures occurs. The **DoubleTapped** event will be used to reset the zoom to the full frame.

C#

```
_mediaPlayerElement.ManipulationMode = ManipulationModes.Scale |  
ManipulationModes.TranslateX | ManipulationModes.TranslateY;  
_mediaPlayerElement.ManipulationDelta +=  
_mediaPlayerElement_ManipulationDelta;  
_mediaPlayerElement.DoubleTapped += _mediaPlayerElement_DoubleTapped;
```

Next, declare a **Rect** object that will store the current zoom source rectangle.

C#

```
Rect _sourceRect = new Rect(0, 0, 1, 1);
```

The **ManipulationDelta** handler adjusts either the scale or the translation of the zoom rectangle. If the delta scale value is not 1, it means that the user performed a pinch gesture. If the value is greater than 1, the source rectangle should be made smaller to zoom into the content. If the value is less than 1, then the source rectangle should be made bigger to zoom out. Before setting the new scale values, the resulting rectangle is checked to make sure it lies entirely within the (0,0,1,1) limits.

If the scale value is 1, then the translation gesture is handled. The rectangle is simply translated by the number of pixels in gesture divided by the width and height of the control. Again, the resulting rectangle is checked to make sure it lies within the (0,0,1,1) bounds.

Finally, the **NormalizedSourceRect** of the **MediaPlaybackSession** is set to the newly adjusted rectangle, specifying the area within the video frame that should be rendered.

C#

```
private void _mediaPlayerElement_ManipulationDelta(object sender,
ManipulationDeltaRoutedEventArgs e)
{
    if (e.Delta.Scale != 1)
    {
        var halfWidth = _sourceRect.Width / 2;
        var halfHeight = _sourceRect.Height / 2;

        var centerX = _sourceRect.X + halfWidth;
        var centerY = _sourceRect.Y + halfHeight;

        var scale = e.Delta.Scale;
        var newHalfWidth = (_sourceRect.Width * e.Delta.Scale) / 2;
        var newHalfHeight = (_sourceRect.Height * e.Delta.Scale) / 2;

        if (centerX - newHalfWidth > 0 && centerX + newHalfWidth <= 1.0 &&
            centerY - newHalfHeight > 0 && centerY + newHalfHeight <= 1.0)
        {
            _sourceRect.X = centerX - newHalfWidth;
            _sourceRect.Y = centerY - newHalfHeight;
            _sourceRect.Width *= e.Delta.Scale;
            _sourceRect.Height *= e.Delta.Scale;
        }
    }
    else
    {
        var translateX = -1 * e.Delta.Translation.X /
_mediaPlayerElement.ActualWidth;
        var translateY = -1 * e.Delta.Translation.Y /
_mediaPlayerElement.ActualHeight;

        if (_sourceRect.X + translateX >= 0 && _sourceRect.X +
_sourceRect.Width + translateX <= 1.0 &&
            _sourceRect.Y + translateY >= 0 && _sourceRect.Y +
_sourceRect.Height + translateY <= 1.0)
        {
            _sourceRect.X += translateX;
            _sourceRect.Y += translateY;
        }
    }

    mediaPlayer.PlaybackSession.NormalizedSourceRect = _sourceRect;
}
```

In the [DoubleTapped](#) event handler, the source rectangle is set back to (0,0,1,1) to cause the entire video frame to be rendered.

C#

```
private void _mediaPlayerElement_DoubleTapped(object sender,
DoubleTappedRoutedEventArgs e)
{
    _sourceRect = new Rect(0, 0, 1, 1);
    mediaPlayer.PlaybackSession.NormalizedSourceRect = _sourceRect;
}
```

NOTE This section describes touch input. Touchpad sends pointer events and will not send Manipulation events.

Handling policy-based playback degradation

In some circumstances the system may degrade the playback of a media item, such as reducing the resolution (constriction), based on a policy rather than a performance issue. For example, video may be degraded by the system if it is being played using an unsigned video driver. You can call

[MediaPlaybackSession.GetOutputDegradationPolicyState](#) to determine if and why this policy-based degradation is occurring and alert the user or record the reason for telemetry purposes.

The following example shows an implementation of a handler for the **MediaPlayer.MediaOpened** event that is raised when the player opens a new media item. **GetOutputDegradationPolicyState** is called on the **MediaPlayer** passed into the handler. The value of [VideoConstrictionReason](#) indicates the policy reason that the video is constricted. If the value isn't **None**, this example logs the degradation reason for telemetry purposes. This example also shows setting the bitrate of the **AdaptiveMediaSource** currently being played to the lowest bandwidth to save data usage, since the video is constricted and won't be displayed at high resolution anyway. For more information on using **AdaptiveMediaSource**, see [Adaptive streaming](#).

C#

```
private void MediaPlayer_MediaOpened(MediaPlayer sender, object args)
{
    MediaPlaybackSessionOutputDegradationPolicyState info =
sender.PlaybackSession.GetOutputDegradationPolicyState();

    if (info.VideoConstrictionReason !=
MediaPlaybackSessionVideoConstrictionReason.None)
    {
        // Switch to lowest bitrate to save bandwidth
        adaptiveMediaSource.DesiredMaxBitrate =
adaptiveMediaSource.AvailableBitrates[0];

        // Log the degradation reason or show a message to the user
    }
}
```

```
System.Diagnostics.Debug.WriteLine("Logging constriction reason: " +
    info.VideoConstrictionReason);
    }
}
```

Use MediaPlayerSurface to render video to a Windows.UI.Composition surface

Starting with Windows 10, version 1607, you can use **MediaPlayer** to render video to an **ICompositionSurface**, which allows the player to interoperate with the APIs in the **Windows.UI.Composition** namespace. The composition framework allows you to work with graphics in the visual layer between XAML and the low-level DirectX graphics APIs. This enables scenarios like rendering video into any XAML control. For more information on using the composition APIs, see [Visual Layer](#).

The following example illustrates how to render video player content onto a **Canvas** control. The media player-specific calls in this example are **SetSurfaceSize** and **GetSurface**. **SetSurfaceSize** tells the system the size of the buffer that should be allocated for rendering content. **GetSurface** takes a **Compositor** as an argument and retrieves an instance of the **MediaPlayerSurface** class. This class provides access to the **MediaPlayer** and **Compositor** used to create the surface and exposes the surface itself through the **CompositionSurface** property.

The rest of the code in this example creates a **SpriteVisual** to which the video is rendered and sets the size to the size of the canvas element that will display the visual. Next a **CompositionBrush** is created from the **MediaPlayerSurface** and assigned to the **Brush** property of the visual. Next a **ContainerVisual** is created and the **SpriteVisual** is inserted at the top of its visual tree. Finally, **SetElementChildVisual** is called to assign the container visual to the **Canvas**.

C#

```
mediaPlayer.SetSurfaceSize(new Size(_compositionCanvas.ActualWidth,
    _compositionCanvas.ActualHeight));

var compositor =
    ElementCompositionPreview.GetElementVisual(this).Compositor;
MediaPlayerSurface surface = mediaPlayer.GetSurface(compositor);

SpriteVisual spriteVisual = compositor.CreateSpriteVisual();
spriteVisual.Size =
    new System.Numerics.Vector2((float)_compositionCanvas.ActualWidth,
    (float)_compositionCanvas.ActualHeight);

CompositionBrush brush =
```

```
compositor.CreateSurfaceBrush(surface.CompositionSurface);
spriteVisual.Brush = brush;

ContainerVisual container = compositor.CreateContainerVisual();
container.Children.InsertAtTop(spriteVisual);

ElementCompositionPreview.SetElementChildVisual(_compositionCanvas,
container);
```

Use MediaTimelineController to synchronize content across multiple players.

As discussed previously in this article, your app can have several **MediaPlayer** objects active at a time. By default, each **MediaPlayer** you create operates independently. For some scenarios, such as synchronizing a commentary track to a video, you may want to synchronize the player state, playback position, and playback speed of multiple players. Starting with Windows 10, version 1607, you can implement this behavior by using the [MediaTimelineController](#) class.

Implement playback controls

The following example shows how to use a **MediaTimelineController** to control two instances of **MediaPlayer**. First, each instance of the **MediaPlayer** is instantiated and the **Source** is set to a media file. Next, a new **MediaTimelineController** is created. For each **MediaPlayer**, the [MediaPlaybackCommandManager](#) associated with each player is disabled by setting the [IsEnabled](#) property to false. And then the [TimelineController](#) property is set to the timeline controller object.

C#

```
MediaTimelineController _mediaTimelineController;
```

C#

```
mediaPlayer = new MediaPlayer();
mediaPlayer.Source = MediaSource.CreateFromUri(new Uri("ms-
appx:///Assets/example_video.mkv"));
_mediaPlayerElement.SetMediaPlayer(mediaPlayer);

_mediaPlayer2 = new MediaPlayer();
_mediaPlayer2.Source = MediaSource.CreateFromUri(new Uri("ms-
appx:///Assets/example_video_2.mkv"));
_mediaPlayerElement2.SetMediaPlayer(_mediaPlayer2);
```

```
_mediaTimelineController = new MediaTimelineController();

mediaPlayer.CommandManager.IsEnabled = false;
mediaPlayer.TimelineController = _mediaTimelineController;

_mediaPlayer2.CommandManager.IsEnabled = false;
_mediaPlayer2.TimelineController = _mediaTimelineController;
```

Caution The [MediaPlaybackCommandManager](#) provides automatic integration between **MediaPlayer** and the System Media Transport Controls (SMTTC), but this automatic integration can't be used with media players that are controlled with a **MediaTimelineController**. Therefore you must disable the command manager for the media player before setting the player's timeline controller. Failure to do so will result in an exception being thrown with the following message: "Attaching Media Timeline Controller is blocked because of the current state of the object." For more information on media player integration with the SMTTC, see [Integrate with the System Media Transport Controls](#). If you are using a **MediaTimelineController** you can still control the SMTTC manually. For more information, see [Manual control of the System Media Transport Controls](#).

Once you have attached a **MediaTimelineController** to one or more media players, you can control the playback state by using the methods exposed by the controller. The following example calls [Start](#) to begin playback of all associated media players at the beginning of the media.

C#

```
private void PlayButton_Click(object sender, RoutedEventArgs e)
{
    _mediaTimelineController.Start();
}
```

This example illustrates pausing and resuming all of the attached media players.

C#

```
private void PauseButton_Click(object sender, RoutedEventArgs e)
{
    if(_mediaTimelineController.State ==
MediaTimelineControllerState.Running)
    {
        _mediaTimelineController.Pause();
        _pauseButton.Content = "Resume";
    }
    else
    {
```

```

        _mediaTimelineController.Resume();
        _pauseButton.Content = "Pause";
    }
}

```

To fast-forward all connected media players, set the playback speed to a value greater than 1.

C#

```

private void FastForwardButton_Click(object sender, RoutedEventArgs e)
{
    _mediaTimelineController.ClockRate = 2.0;
}

```

The next example shows how to use a **Slider** control to show the current playback position of the timeline controller relative to the duration of the content of one of the connected media players. First, a new **MediaSource** is created and a handler for the **OpenOperationCompleted** of the media source is registered.

C#

```

var mediaSource = MediaSource.CreateFromUri(new Uri("ms-appx:///Assets/example_video.mkv"));
mediaSource.OpenOperationCompleted += MediaSource_OpenOperationCompleted;
mediaPlayer.Source = mediaSource;
_mediaPlayerElement.SetMediaPlayer(mediaPlayer);

```

The **OpenOperationCompleted** handler is used as an opportunity to discover the duration of the media source content. Once the duration is determined, the maximum value of the **Slider** control is set to the total number of seconds of the media item. The value is set inside a call to **RunAsync** to make sure it is run on the UI thread.

C#

```

    TimeSpan _duration;

```

C#

```

private async void MediaSource_OpenOperationCompleted(MediaSource sender,
    MediaSourceOpenOperationCompletedEventArgs args)
{
    _duration = sender.Duration.GetValueOrDefault();

    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>

```

```

    {
        _positionSlider.Minimum = 0;
        _positionSlider.Maximum = _duration.TotalSeconds;
        _positionSlider.StepFrequency = 1;
    });
}

```

Next, a handler for the timeline controller's **PositionChanged** event is registered. This is called periodically by the system, approximately 4 times per second.

C#

```

_mediaTimelineController.PositionChanged +=
_mediaTimelineController_PositionChanged;

```

In the handler for **PositionChanged**, the slider value is updated to reflect the current position of the timeline controller.

C#

```

private async void
_mediaTimelineController_PositionChanged(MediaTimelineController sender,
object args)
{
    if (_duration != TimeSpan.Zero)
    {
        await
Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal, () =>
        {
            _positionSlider.Value = sender.Position.TotalSeconds /
(float)_duration.TotalSeconds;
        });
    }
}

```

Offset the playback position from the timeline position

In some cases you may want the playback position of one or more media players associated with a timeline controller to be offset from the other players. You can do this by setting the **TimelineControllerPositionOffset** property of the **MediaPlayer** object you want to be offset. The following example uses the durations of the content of two media players to set the minimum and maximum values of two slider control to plus and minus the length of the item.

C#

```
_timelineOffsetSlider1.Minimum = -1 * _duration.TotalSeconds;  
_timelineOffsetSlider1.Maximum = _duration.TotalSeconds;  
_timelineOffsetSlider1.StepFrequency = 1;  
  
_timelineOffsetSlider2.Minimum = -1 * _duration2.TotalSeconds;  
_timelineOffsetSlider2.Maximum = _duration2.TotalSeconds;  
_timelineOffsetSlider2.StepFrequency = 1;
```

In the **ValueChanged** event for each slider, the **TimelineControllerPositionOffset** for each player is set to the corresponding value.

C#

```
private void _timelineOffsetSlider1_ValueChanged(object sender,  
RangeBaseValueChangedEventArgs e)  
{  
    mediaPlayer.TimelineControllerPositionOffset =  
    TimeSpan.FromSeconds(_timelineOffsetSlider1.Value);  
}  
  
private void _timelineOffsetSlider2_ValueChanged(object sender,  
RangeBaseValueChangedEventArgs e)  
{  
    _mediaPlayer2.TimelineControllerPositionOffset =  
    TimeSpan.FromSeconds(_timelineOffsetSlider2.Value);  
}
```

Note that if the offset value of a player maps to a negative playback position, the clip will remain paused until the offset reaches zero and then playback will begin. Likewise, if the offset value maps to a playback position greater than the duration of the media item, the final frame will be shown, just as it does when a single media player reached the end of its content.

Play spherical video with MediaPlayer

Starting with Windows 10, version 1703, **MediaPlayer** supports equirectangular projection for spherical video playback. Spherical video content is no different from regular, flat video in that **MediaPlayer** will render the video as long as the video encoding is supported. For spherical video that contains a metadata tag that specifies that the video uses equirectangular projection, **MediaPlayer** can render the video using a specified field-of-view and view orientation. This enables scenarios such as virtual reality video playback with a head-mounted display or simply allowing the user to pan around within spherical video content using the mouse or keyboard input.

To play back spherical video, use the steps for playing back video content described previously in this article. The one additional step is to register a handler for the [MediaPlayer.MediaOpened](#) event. This event gives you an opportunity to enable and control the spherical video playback parameters.

C#

```
mediaPlayer = new MediaPlayer();
mediaPlayer.MediaOpened += _mediaPlayer_MediaOpened;
mediaPlayer.Source = MediaSource.CreateFromUri(new Uri("ms-appx:///Assets/example_video_spherical.mp4"));
_mediaPlayerElement.SetMediaPlayer(mediaPlayer);
mediaPlayer.Play();
```

In the **MediaOpened** handler, first check the frame format of the newly opened media item by checking the [PlaybackSession.SphericalVideoProjection.FrameFormat](#) property. If this value is [SphericalVideoFrameFormat.Equirectangular](#), then the system can automatically project the video content. First, set the [PlaybackSession.SphericalVideoProjection.IsEnabled](#) property to **true**. You can also adjust properties such as the view orientation and field of view that the media player will use to project the video content. In this example, the field of view is set to a wide value of 120 degrees by setting the [HorizontalFieldOfViewInDegrees](#) property.

If the video content is spherical, but is in a format other than equirectangular, you can implement your own projection algorithm using the media player's frame server mode to receive and process individual frames.

C#

```
private void _mediaPlayer_MediaOpened(MediaPlayer sender, object args)
{
    if (sender.PlaybackSession.SphericalVideoProjection.FrameFormat ==
        SphericalVideoFrameFormat.Equirectangular)
    {
        sender.PlaybackSession.SphericalVideoProjection.IsEnabled = true;

        sender.PlaybackSession.SphericalVideoProjection.HorizontalFieldOfViewInDegrees = 120;
    }
    else if (sender.PlaybackSession.SphericalVideoProjection.FrameFormat ==
        SphericalVideoFrameFormat.Unsupported)
    {
        // If the spherical format is unsupported, you can use frame server
        mode to implement a custom projection
    }
}
```

The following example code illustrates how to adjust the spherical video view orientation using the left and right arrow keys.

C#

```
protected override void OnKeyDown(KeyRoutedEventArgs e)
{
    if (mediaPlayer.PlaybackSession.SphericalVideoProjection.FrameFormat !=
        SphericalVideoFrameFormat.Equirectangular)
    {
        return;
    }

    switch (e.Key)
    {
        case Windows.System.VirtualKey.Right:

            mediaPlayer.PlaybackSession.SphericalVideoProjection.ViewOrientation *=
                Quaternion.CreateFromYawPitchRoll(.1f, 0, 0);
            break;
        case Windows.System.VirtualKey.Left:

            mediaPlayer.PlaybackSession.SphericalVideoProjection.ViewOrientation *=
                Quaternion.CreateFromYawPitchRoll(-.1f, 0, 0);
            break;
    }
}
```

If your app supports playlists of video, you may want to identify playback items that contain spherical video in your UI. Media playlists are discussed in detail in the article, [Media items, playlists, and tracks](#). The following example shows creating a new playlist, adding an item, and registering a handler for the [MediaPlaybackItem.VideoTracksChanged](#) event, which occurs when the video tracks for a media item are resolved.

C#

```
var playbackList = new MediaPlaybackList();
var item = new MediaPlaybackItem(MediaSource.CreateFromUri(new Uri("ms-appx:///Assets/RIFTCOASTER HD_injected.mp4")));
item.VideoTracksChanged += Item_VideoTracksChanged;
playbackList.Items.Add(item);
mediaPlayer.Source = playbackList;
```

In the **VideoTracksChanged** event handler, get the encoding properties for any added video tracks by calling [VideoTrack.GetEncodingProperties](#). If the [SphericalVideoFrameFormat](#) property of the encoding properties is a value other than

[SphericalVideoFrameFormat.None](#), then the video track contains spherical video and you can update your UI accordingly if you choose.

C#

```
private void Item_VideoTracksChanged(MediaPlaybackItem sender,
IVectorChangedEventArgs args)
{
    if (args.CollectionChange != CollectionChange.ItemInserted)
    {
        return;
    }
    foreach (var videoTrack in sender.VideoTracks)
    {
        if (videoTrack.GetEncodingProperties().SphericalVideoFrameFormat !=
SphericalVideoFrameFormat.None)
        {
            // Optionally indicate in the UI that this item contains
spherical video
        }
    }
}
```

Use MediaPlayer in frame server mode

Starting with Windows 10, version 1703, you can use **MediaPlayer** in frame server mode. In this mode, the **MediaPlayer** does not automatically render frames to an associated **MediaPlayerElement**. Instead, your app copies the current frame from the **MediaPlayer** to an object that implements [IDirect3DSurface](#). The primary scenario this feature enables is using pixel shaders to process video frames provided by the **MediaPlayer**. Your app is responsible for displaying each frame after processing, such as by showing the frame in a XAML [Image](#) control.

In the following example, a new **MediaPlayer** is initialized and video content is loaded. Next, a handler for [VideoFrameAvailable](#) is registered. Frame server mode is enabled by setting the **MediaPlayer** object's [IsVideoFrameServerEnabled](#) property to **true**. Finally, media playback is started with a call to [Play](#).

C#

```
mediaPlayer = new MediaPlayer();
mediaPlayer.Source = MediaSource.CreateFromUri(new Uri("ms-
appx:///Assets/example_video.mkv"));
mediaPlayer.VideoFrameAvailable += mediaPlayer_VideoFrameAvailable;
mediaPlayer.IsVideoFrameServerEnabled = true;
mediaPlayer.Play();
```

The next example shows a handler for **VideoFrameAvailable** that uses [Win2D](#) to add a simple blur effect to each frame of a video and then displays the processed frames in a XAML **Image** control.

Whenever the **VideoFrameAvailable** handler is called, the **CopyFrameToVideoSurface** method is used to copy the contents of the frame to an **IDirect3DSurface**. You can also use **CopyFrameToStereoscopicVideoSurfaces** to copy 3D content into two surfaces, for processing the left eye and right eye content separately. To get an object that implements **IDirect3DSurface** this example creates a **SoftwareBitmap** and then uses that object to create a Win2D **CanvasBitmap**, which implements the necessary interface. A **CanvasImageSource** is a Win2D object that can be used as the source for an **Image** control, so a new one is created and set as the source for the **Image** in which the content will be displayed. Next, a **CanvasDrawingSession** is created. This is used by Win2D to render the blur effect.

Once all of the necessary objects have been instantiated, **CopyFrameToVideoSurface** is called, which copies the current frame from the **MediaPlayer** into the **CanvasBitmap**. Next, a Win2D **GaussianBlurEffect** is created, with the **CanvasBitmap** set as the source of the operation. Finally, **CanvasDrawingSession.DrawImage** is called to draw the source image, with the blur effect applied, into the **CanvasImageSource** that has been associated with **Image** control, causing it to be drawn in the UI.

C#

```
private async void mediaPlayer_VideoFrameAvailable(MediaPlayer sender,
object args)
{
    CanvasDevice canvasDevice = CanvasDevice.GetSharedDevice();

    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        if(frameServerDest == null)
        {
            // FrameServerImage in this example is a XAML image control
            frameServerDest = new SoftwareBitmap(BitmapPixelFormat.Rgba8,
            (int)FrameServerImage.Width, (int)FrameServerImage.Height,
            BitmapAlphaMode.Ignore);
        }
        if(canvasImageSource == null)
        {
            canvasImageSource = new CanvasImageSource(canvasDevice,
            (int)FrameServerImage.Width, (int)FrameServerImage.Height,
            DisplayInformation.GetForCurrentView().LogicalDpi); //96);
            FrameServerImage.Source = canvasImageSource;
        }

        using (CanvasBitmap inputBitmap =
```

```

CanvasBitmap.CreateFromSoftwareBitmap(canvasDevice, frameServerDest))
    using (CanvasDrawingSession ds =
canvasImageSource.CreateDrawingSession(Windows.UI.Colors.Black))
    {

        mediaPlayer.CopyFrameToVideoSurface(inputBitmap);

        var gaussianBlurEffect = new GaussianBlurEffect
        {
            Source = inputBitmap,
            BlurAmount = 5f,
            Optimization = EffectOptimization.Speed
        };

        ds.DrawImage(gaussianBlurEffect);

    }
});
}

private void FrameServerSubtitlesButton_Click(object sender, RoutedEventArgs
e)
{

    mediaPlayer = new MediaPlayer();
    var source = MediaSource.CreateFromUri(new Uri("ms-
appx:///Assets/example_video.mkv"));
    var item = new MediaPlaybackItem(source);

    item.TimedMetadataTracksChanged += Item_TimedMetadataTracksChanged;

    mediaPlayer.Source = item;
    mediaPlayer.VideoFrameAvailable +=
mediaPlayer_VideoFrameAvailable_Subtitle;
    mediaPlayer.IsVideoFrameServerEnabled = true;
    mediaPlayer.Play();

    mediaPlayer.IsMuted = true;

}

private void Item_TimedMetadataTracksChanged(MediaPlaybackItem sender,
IVectorChangedEventArgs args)
{
    if(sender.TimedMetadataTracks.Count > 0)
    {
        sender.TimedMetadataTracks.SetPresentationMode(0,
TimedMetadataTrackPresentationMode.PlatformPresented);
    }
}

private async void mediaPlayer_VideoFrameAvailable_Subtitle(MediaPlayer
sender, object args)
{

```

```

CanvasDevice canvasDevice = CanvasDevice.GetSharedDevice();

await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
()) =>
{
    if (frameServerDest == null)
    {
        // FrameServerImage in this example is a XAML image control
        frameServerDest = new SoftwareBitmap(BitmapPixelFormat.Rgba8,
(int)frameServerImage.Width, (int)frameServerImage.Height,
BitmapAlphaMode.Ignore);
    }
    if (canvasImageSource == null)
    {
        canvasImageSource = new CanvasImageSource(canvasDevice,
(int)frameServerImage.Width, (int)frameServerImage.Height,
DisplayInformation.GetForCurrentView().LogicalDpi); //96);
        FrameServerImage.Source = canvasImageSource;
    }

    using (CanvasBitmap inputBitmap =
CanvasBitmap.CreateFromSoftwareBitmap(canvasDevice, frameServerDest))
    {
        using (CanvasDrawingSession ds =
canvasImageSource.CreateDrawingSession(Windows.UI.Colors.Black))
        {

            mediaPlayer.CopyFrameToVideoSurface(inputBitmap);

            //Rect subtitleTargetRect = new Rect(0, 0,
inputBitmap.Bounds.Width, inputBitmap.Bounds.Bottom * .1);
            Rect subtitleTargetRect = new Rect(0, 0, 100, 100);

            mediaPlayer.RenderSubtitlesToSurface(inputBitmap); //,
subtitleTargetRect);

            //var gaussianBlurEffect = new GaussianBlurEffect
            //{
            //    Source = inputBitmap,
            //    BlurAmount = 5f,
            //    Optimization = EffectOptimization.Speed
            //};

            //ds.DrawImage(gaussianBlurEffect);

            ds.DrawImage(inputBitmap);
        }
    }
});
}

```

For more information on Win2D, see the [Win2D GitHub repository](#). To try out the sample code shown above, you will need to add the Win2D NuGet package to your

project with the following instructions.

To add the Win2D NuGet package to your effect project

1. In **Solution Explorer**, right-click your project and select **Manage NuGet Packages**.
2. At the top of the window, select the **Browse** tab.
3. In the search box, enter **Win2D**.
4. Select **Win2D.uwp**, and then select **Install** in the right pane.
5. The **Review Changes** dialog shows you the package to be installed. Click **OK**.
6. Accept the package license.

Detect and respond to audio level changes by the system

Starting with Windows 10, version 1803, your app can detect when the system lowers or mutes the audio level of a currently playing **MediaPlayer**. For example, the system may lower, or "duck", the audio playback level when an alarm is ringing. The system will mute your app when it goes into the background if your app has not declared the *backgroundMediaPlayback* capability in the app manifest. The [AudioStateMonitor](#) class allows you to register to receive an event when the system modifies the volume of an audio stream. Access the **AudioStateMonitor** property of a **MediaPlayer** and register a handler for the [SoundLevelChanged](#) event to be notified when the audio level for that **MediaPlayer** is changed by the system.

C#

```
mediaPlayer.AudioStateMonitor.SoundLevelChanged +=  
AudioStateMonitor_SoundLevelChanged;
```

When handling the **SoundLevelChanged** event, you may take different actions depending on the type of content being played. If you are currently playing music, then you may want to let the music continue to play while the volume is ducked. If you are playing a podcast, however, you likely want to pause playback while the audio is ducked so the user doesn't miss any of the content.

This example declares a variable to track whether the currently playing content is a podcast, it is assumed that you set this to the appropriate value when selecting the content for the **MediaPlayer**. We also create a class variable to track when we pause playback programmatically when the audio level changes.

C#

```
bool isPodcast;  
bool isPausedDueToAudioStateMonitor;
```

In the **SoundLevelChanged** event handler, check the **SoundLevel** property of the **AudioStateMonitor** sender to determine the new sound level. This example checks to see if the new sound level is full volume, meaning the system has stopped muting or ducking the volume, or if the sound level has been lowered but is playing non-podcast content. If either of these are true and the content was previously paused programmatically, playback is resumed. If the new sound level is muted or if the current content is a podcast and the sound level is low, playback is paused, and the variable is set to track that the pause was initiated programmatically.

C#

```
private void  
AudioStateMonitor_SoundLevelChanged(Windows.Media.Audio.AudioStateMonitor  
sender, object args)  
{  
    if ((sender.SoundLevel == SoundLevel.Full) || (sender.SoundLevel ==  
SoundLevel.Low && !isPodcast))  
    {  
        if (isPausedDueToAudioStateMonitor)  
        {  
            mediaPlayer.Play();  
            isPausedDueToAudioStateMonitor = false;  
        }  
    }  
    else if ((sender.SoundLevel == SoundLevel.Muted) ||  
            (sender.SoundLevel == SoundLevel.Low && isPodcast))  
    {  
        if (mediaPlayer.PlaybackSession.PlaybackState ==  
MediaPlaybackState.Playing)  
        {  
            mediaPlayer.Pause();  
            isPausedDueToAudioStateMonitor = true;  
        }  
    }  
}
```

The user may decide that they want to pause or continue playback, even if the audio is ducked by the system. This example shows event handlers for a play and a pause button. In the pause button click handler is paused, if playback had already been paused programmatically, then we update the variable to indicate that the user has paused the content. In the play button click handler, we resume playback and clear our tracking variable.

C#

```
private void PauseButton_User_Click(object sender, RoutedEventArgs e)
{
    if (isPausedDueToAudioStateMonitor)
    {
        isPausedDueToAudioStateMonitor = false;
    }
    else
    {
        mediaPlayer.Pause();
    }
}

public void PlayButton_User_Click()
{
    isPausedDueToAudioStateMonitor = false;
    mediaPlayer.Play();
}
```

Related topics

- [Media playback](#)
- [Media items, playlists, and tracks](#)
- [Integrate with the System Media Transport Controls](#)
- [Create, schedule, and manage media breaks](#)
- [Play media in the background](#)

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Media items, playlists, and tracks

Article • 10/20/2022

This article shows you how to use the [MediaSource](#) class, which provides a common way to reference and play back media from different sources such as local or remote files and exposes a common model for accessing media data, regardless of the underlying media format. The [MediaPlaybackItem](#) class extends the functionality of **MediaSource**, allowing you to manage and select from multiple audio, video, and metadata tracks contained in a media item. [MediaPlaybackList](#) allows you to create playback lists from one or more media playback items.

Create and play a MediaSource

Create a new instance of **MediaSource** by calling one of the factory methods exposed by the class:

- [CreateFromAdaptiveMediaSource](#)
- [CreateFromIMediaSource](#)
- [CreateFromMediaStreamSource](#)
- [CreateFromMseStreamSource](#)
- [CreateFromStorageFile](#)
- [CreateFromStream](#)
- [CreateFromStreamReference](#)
- [CreateFromUri](#)
- [CreateFromDownloadOperation](#)

After creating a **MediaSource** you can play it with a [MediaPlayer](#) by setting the [Source](#) property. Starting with Windows 10, version 1607, you can assign a **MediaPlayer** to a [MediaPlayerElement](#) by calling [SetMediaPlayer](#) in order to render the media player content in a XAML page. This is the preferred method over using **MediaElement**. For more information on using **MediaPlayer**, see [Play audio and video with MediaPlayer](#).

The following example shows how to play back a user-selected media file in a **MediaPlayer** using **MediaSource**.

You will need to include the [Windows.Media.Core](#) and [Windows.Media.Playback](#) namespaces in order to complete this scenario.

C#

```
using Windows.Media.Core;
```

```
using Windows.Media.Playback;
```

Declare a variable of type **MediaSource**. For the examples in this article, the media source is declared as a class member so that it can be accessed from multiple locations.

C#

```
MediaSource _mediaSource;
```

Declare a variable to store the **MediaPlayer** object and, if you want to render the media content in XAML, add a **MediaPlayerElement** control to your page.

C#

```
MediaPlayer _mediaPlayer;
```

XML

```
<MediaPlayerElement x:Name="mediaPlayerElement"/>
```

To allow the user to pick a media file to play, use a [FileOpenPicker](#). With the [StorageFile](#) object returned from the picker's [PickSingleFileAsync](#) method, initialize a new [MediaObject](#) by calling [MediaSource.CreateFromStorageFile](#). Finally, set the media source as the playback source for the **MediaElement** by calling the [SetPlaybackSource](#) method.

C#

```
//Create a new picker
var filePicker = new Windows.Storage.Pickers.FileOpenPicker();

//make a collection of all video types you want to support (for testing we
are adding just 3).
string[] fileTypes = new string[] { ".wmv", ".mp4", ".mkv" };

//Add your fileTypes to the FileTypeFilter list of filePicker.
foreach (string fileType in fileTypes)
{
    filePicker.FileTypeFilter.Add(fileType);
}

//Set picker start location to the video library
filePicker.SuggestedStartLocation = PickerLocationId.VideosLibrary;

//Retrieve file from picker
StorageFile file = await filePicker.PickSingleFileAsync();
```

```
if (!(file is null))
{
    _mediaSource = MediaSource.CreateFromStorageFile(file);
    _mediaPlayer = new MediaPlayer();
    _mediaPlayer.Source = _mediaSource;
    mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
}
```

By default, the **MediaPlayer** does not begin playing automatically when the media source is set. You can manually begin playback by calling [Play](#).

C#

```
_mediaPlayer.Play();
```

You can also set the [AutoPlay](#) property of the **MediaPlayer** to true to tell the player to begin playing as soon as the media source is set.

C#

```
_mediaPlayer.AutoPlay = true;
```

Create a MediaSource from a DownloadOperation

Starting with Windows, version 1803, you can create a **MediaSource** object from a **DownloadOperation**.

C#

```
StorageFile destinationFile = await
KnownFolders.VideosLibrary.CreateFileAsync("file.mp4",
CreationCollisionOption.GenerateUniqueName);

var downloader = new BackgroundDownloader();
var downloadOperation = downloader.CreateDownload(new
Uri("http://server.com/file.mp4"), destinationFile);
MediaSource mediaSource =
    MediaSource.CreateFromDownloadOperation(downloadOperation);
```

Note that while you can create a **MediaSource** from a download without starting it or setting its **IsRandomAccessRequired** property to true, you must do both of these things before attempting to attach the **MediaSource** to a **MediaPlayer** or **MediaPlayerElement** for playback.

C#

```
downloadOperation.IsRandomAccessRequired = true;  
var startAsyncTask = downloadOperation.StartAsync().AsTask();  
mediaPlayerElement.Source = mediaSource;
```

Handle multiple audio, video, and metadata tracks with `MediaPlaybackItem`

Using a `MediaSource` for playback is convenient because it provides a common way to playback media from different kinds of sources, but more advanced behavior can be accessed by creating a `MediaPlaybackItem` from the `MediaSource`. This includes the ability to access and manage multiple audio, video, and data tracks for a media item.

Declare a variable to store your `MediaPlaybackItem`.

C#

```
MediaPlaybackItem _mediaPlaybackItem;
```

Create a `MediaPlaybackItem` by calling the constructor and passing in an initialized `MediaSource` object.

If your app supports multiple audio, video, or data tracks in a media playback item, register event handlers for the `AudioTracksChanged`, `VideoTracksChanged`, or `TimedMetadataTracksChanged` events.

Finally, set the playback source of the `MediaElement` or `MediaPlayer` to your `MediaPlaybackItem`.

C#

```
_mediaSource = MediaSource.CreateFromStorageFile(file);  
_mediaPlaybackItem = new MediaPlaybackItem(_mediaSource);  
  
_mediaPlaybackItem.AudioTracksChanged += PlaybackItem_AudioTracksChanged;  
_mediaPlaybackItem.VideoTracksChanged +=  
MediaPlaybackItem_VideoTracksChanged;  
_mediaPlaybackItem.TimedMetadataTracksChanged +=  
MediaPlaybackItem_TimedMetadataTracksChanged;  
  
_mediaPlayer = new MediaPlayer();  
_mediaPlayer.Source = _mediaPlaybackItem;  
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
```

ⓘ Note

A **MediaSource** can only be associated with a single **MediaPlaybackItem**. After creating a **MediaPlaybackItem** from a source, attempting to create another playback item from the same source will result in an error. Also, after creating a **MediaPlaybackItem** from a media source, you can't set the **MediaSource** object directly as the source for a **MediaPlayer** but should instead use the **MediaPlaybackItem**.

The **VideoTracksChanged** event is raised after a **MediaPlaybackItem** containing multiple video tracks is assigned as a playback source, and can be raised again if the list of video tracks changes for the item changes. The handler for this event gives you the opportunity to update your UI to allow the user to switch between available tracks. This example uses a **ComboBox** to display the available video tracks.

XML

```
<ComboBox x:Name="videoTracksComboBox"
SelectionChanged="videoTracksComboBox_SelectionChanged"/>
```

In the **VideoTracksChanged** handler, loop through all of the tracks in the playback item's **VideoTracks** list. For each track, a new **ComboBoxItem** is created. If the track does not already have a label, a label is generated from the track index. The **Tag** property of the combo box item is set to the track index so that it can be identified later. Finally, the item is added to the combo box. Note that these operations are performed within a **CoreDispatcher.RunAsync** call because all UI changes must be made on the UI thread and this event is raised on a different thread.

C#

```
private async void MediaPlayerItem_VideoTracksChanged(MediaPlaybackItem
sender, IVectorChangedEventArgs args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        videoTracksComboBox.Items.Clear();
        for (int index = 0; index < sender.VideoTracks.Count; index++)
        {
            var videoTrack = sender.VideoTracks[index];
            ComboBoxItem item = new ComboBoxItem();
            item.Content = String.IsNullOrEmpty(videoTrack.Label) ? $"Track
{index}" : videoTrack.Label;
            item.Tag = index;
            videoTracksComboBox.Items.Add(item);
        }
    });
}
```

```

    }
    });
}

```

In the [SelectionChanged](#) handler for the combo box, the track index is retrieved from the selected item's **Tag** property. Setting the [SelectedIndex](#) property of the media playback item's [VideoTracks](#) list causes the **MediaElement** or **MediaPlayer** to switch the active video track to the specified index.

C#

```

private void videoTracksComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    int trackIndex = (int)((ComboBoxItem)
((ComboBox)sender).SelectedItem).Tag;
    _mediaPlaybackItem.VideoTracks.SelectedIndex = trackIndex;
}

```

Managing media items with multiple audio tracks works exactly the same as with video tracks. Handle the [AudioTracksChanged](#) to update your UI with the audio tracks found in the playback item's [AudioTracks](#) list. When the user selects an audio track, set the [SelectedIndex](#) property of the **AudioTracks** list to cause the **MediaElement** or **MediaPlayer** to switch the active audio track to the specified index.

XML

```

<ComboBox x:Name="audioTracksComboBox"
SelectionChanged="audioTracksComboBox_SelectionChanged"/>

```

C#

```

private async void PlaybackItem_AudioTracksChanged(MediaPlaybackItem sender,
IVectorChangedEventArgs args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
() =>
    {
        audioTracksComboBox.Items.Clear();
        for (int index = 0; index < sender.AudioTracks.Count; index++)
        {
            var audioTrack = sender.AudioTracks[index];
            ComboBoxItem item = new ComboBoxItem();
            item.Content = String.IsNullOrEmpty(audioTrack.Label) ? $"Track
{index}" : audioTrack.Label;
            item.Tag = index;
            videoTracksComboBox.Items.Add(item);
        }
    }
}

```

```
});
}
```

C#

```
private void audioTracksComboBox_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    int trackIndex = (int)((ComboBoxItem)
((ComboBox)sender).SelectedItem).Tag;
    _mediaPlaybackItem.AudioTracks.SelectedIndex = trackIndex;
}
```

In addition to audio and video, a **MediaPlaybackItem** object may contain zero or more **TimedMetadataTrack** objects. A timed metadata track can contain subtitle or caption text, or it may contain custom data that is proprietary to your app. A timed metadata track contains a list of cues represented by objects that inherit from **IMediaCue**, such as a **DataCue** or a **TimedTextCue**. Each cue has a start time and a duration that determines when the cue is activated and for how long.

Similar to audio tracks and video tracks, the timed metadata tracks for a media item can be discovered by handling the **TimedMetadataTracksChanged** event of a **MediaPlaybackItem**. With timed metadata tracks, however, the user may want to enable more than one metadata track at a time. Also, depending on your app scenario, you may want to enable or disable metadata tracks automatically, without user intervention. For illustration purposes, this example adds a **ToggleButton** for each metadata track in a media item to allow the user to enable and disable the track. The **Tag** property of each button is set to the index of the associated metadata track so that it can be identified when the button is toggled.

XML

```
<StackPanel x:Name="MetadataButtonPanel" Orientation="Horizontal"/>
```

C#

```
private async void
MediaPlaybackItem_TimedMetadataTracksChanged(MediaPlaybackItem sender,
IVectorChangedEventArgs args)
{
    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        for (int index = 0; index < sender.TimedMetadataTracks.Count;
index++)
        {
```



```

        var timedMetadataTrack = sender.TimedMetadataTracks[index];

        ToggleButton toggle = new ToggleButton()
        {
            Content = String.IsNullOrEmpty(timedMetadataTrack.Label) ?
            $"Track {index}" : timedMetadataTrack.Label,
            Tag = (uint)index
        };
        toggle.Checked += Toggle_Checked;
        toggle.Unchecked += Toggle_Unchecked;

        MetadataButtonPanel.Children.Add(toggle);
    }
});
}

```

Because more than one metadata track can be active at a time, you don't simply set the active index for the metadata track list. Instead, call the **MediaPlaybackItem** object's **SetPresentationMode** method, passing in the index of the track you want to toggle, and then providing a value from the **TimedMetadataTrackPresentationMode** enumeration. The presentation mode you choose depends on the implementation of your app. In this example, the metadata track is set to **PlatformPresented** when enabled. For text-based tracks, this means that the system will automatically display the text cues in the track. When the toggle button is toggled off, the presentation mode is set to **Disabled**, which means that no text is displayed and no cue events are raised. Cue events are discussed later in this article.

C#

```

private void Toggle_Checked(object sender, RoutedEventArgs e) =>
    _mediaPlaybackItem.TimedMetadataTracks.SetPresentationMode((uint)
        ((ToggleButton)sender).Tag,
        TimedMetadataTrackPresentationMode.PlatformPresented);

```

C#

```

private void Toggle_Unchecked(object sender, RoutedEventArgs e) =>
    _mediaPlaybackItem.TimedMetadataTracks.SetPresentationMode((uint)
        ((ToggleButton)sender).Tag,
        TimedMetadataTrackPresentationMode.Disabled);

```

As you are processing the metadata tracks, you can access the set of cues within the track by accessing the **Cues** or **ActiveCues** properties. You can do this to update your UI to show the cue locations for a media item.

Handle unsupported codecs and unknown errors when opening media items

Starting with Windows 10, version 1607, you can check whether the codec required to playback a media item is supported or partially supported on the device on which your app is running. In the event handler for the **MediaPlaybackItem** tracks-changed events, such as **AudioTracksChanged**, first check to see if the track change is an insertion of a new track. If so, you can get a reference to the track being inserted by using the index passed in the **IVectorChangedEventArgs.Index** parameter with the appropriate track collection of the **MediaPlaybackItem** parameter, such as the **AudioTracks** collection.

Once you have a reference to the inserted track, check the **DecoderStatus** of the track's **SupportInfo** property. If the value is **FullySupported**, then the appropriate codec needed to play back the track is present on the device. If the value is **Degraded**, then the track can be played by the system, but the playback will be degraded in some way. For example, a 5.1 audio track may be played back as 2-channel stereo instead. If this is the case, you may want to update your UI to alert the user of the degradation. If the value is **UnsupportedSubtype** or **UnsupportedEncoderProperties**, then the track can't be played back at all with the current codecs on the device. You may wish to alert the user and skip playback of the item or implement UI to allow the user to download the correct codec. The track's **GetEncodingProperties** method can be used to determine the required codec for playback.

Finally, you can register for the track's **OpenFailed** event, which will be raised if the track is supported on the device but failed to open due to an unknown error in the pipeline.

C#

```
private async void SnippetAudioTracksChanged_CodecCheck(MediaPlaybackItem
sender, IVectorChangedEventArgs args)
{
    if (args.CollectionChange == CollectionChange.ItemInserted)
    {
        var insertedTrack = sender.AudioTracks[(int)args.Index];

        var decoderStatus = insertedTrack.SupportInfo.DecoderStatus;
        if (decoderStatus != MediaDecoderStatus.FullySupported)
        {
            if (decoderStatus == MediaDecoderStatus.Degraded)
            {
                ShowMessageToUser($"Track {insertedTrack.Name} can play but
playback will be degraded. {insertedTrack.SupportInfo.DegradationReason}");
            }
            else
            {
                // status is MediaDecoderStatus.UnsupportedSubtype or
```

```

MediaDecoderStatus.UnsupportedEncoderProperties
    ShowMessageToUser($"Track {insertedTrack.Name} uses an
unsupported media format.");
}

Windows.Media.MediaProperties.AudioEncodingProperties props =
insertedTrack.GetEncodingProperties();
await HelpUserInstallCodec(props);
}
else
{
    insertedTrack.OpenFailed += InsertedTrack_OpenFailed;
}
}
}
}

```

In the **OpenFailed** event handler, you can check to see if the **MediaSource** status is unknown, and if so, you can programmatically select a different track to play, allow the user to choose a different track, or abandon playback.

```

C#

private async void InsertedTrack_OpenFailed(AudioTrack sender,
AudioTrackOpenFailedEventArgs args)
{
    LogError(args.ExtendedError.HResult);

    if (sender.SupportInfo.MediaSourceStatus == MediaSourceStatus.Unknown)
    {
        await SelectAnotherTrackOrSkipPlayback(sender.PlaybackItem);
    }
}
}

```

Set display properties used by the System Media Transport Controls

Starting with Windows 10, version 1607, media played in a **MediaPlayer** is automatically integrated into the System Media Transport Controls (SMTTC) by default. You can specify the metadata that will be displayed by the SMTTC by updating the display properties for a **MediaPlaybackItem**. Get an object representing the display properties for an item by calling **GetDisplayProperties**. Set whether the playback item is music or video by setting the **Type** property. Then, set the properties of the object's **VideoProperties** or **MusicProperties**. Call **ApplyDisplayProperties** to update the item's properties to the values you provided. Typically, an app will retrieve the display values dynamically from a web service, but the following example illustrates this process with hardcoded values.

C#

```
MediaItemDisplayProperties props = mediaPlaybackItem.GetDisplayProperties();
props.Type = Windows.Media.MediaPlaybackType.Video;
props.VideoProperties.Title = "Video title";
props.VideoProperties.Subtitle = "Video subtitle";
props.VideoProperties.Genres.Add("Documentary");
mediaPlaybackItem.ApplyDisplayProperties(props);
```

C#

```
props = mediaPlaybackItem.GetDisplayProperties();
props.Type = Windows.Media.MediaPlaybackType.Music;
props.MusicProperties.Title = "Song title";
props.MusicProperties.Artist = "Song artist";
props.MusicProperties.Genres.Add("Polka");
mediaPlaybackItem.ApplyDisplayProperties(props);
```

Add external timed text with TimedTextSource

For some scenarios, you may have external files that contains timed text associated with a media item, such as separate files that contain subtitles for different locales. Use the [TimedTextSource](#) class to load in external timed text files from a stream or URI.

This example uses a **Dictionary** collection to store a list of the timed text sources for the media item using the source URI and the **TimedTextSource** object as the key/value pair in order to identify the tracks after they have been resolved.

C#

```
Dictionary<TimedTextSource, Uri> timedTextSourceMap;
```

Create a new **TimedTextSource** for each external timed text file by calling [CreateFromUri](#). Add an entry to the **Dictionary** for the timed text source. Add a handler for the [TimedTextSource.Resolved](#) event to handle if the item failed to load or to set additional properties after the item was loaded successfully.

Register all of your **TimedTextSource** objects with the **MediaSource** by adding them to the [ExternalTimedTextSources](#) collection. Note that external timed text sources are added to directly the **MediaSource** and not the **MediaPlaybackItem** created from the source. To update your UI to reflect the external text tracks, register and handle the **TimedMetadataTracksChanged** event as described previously in this article.

C#

```

// Create the TimedTextSource and add entry to URI map
var timedTextSourceUri_En = new
Uri("http://contoso.com/MyClipTimedText_en.srt");
var timedTextSource_En =
TimedTextSource.CreateFromUri(timedTextSourceUri_En);
timedTextSourceMap[timedTextSource_En] = timedTextSourceUri_En;
timedTextSource_En.Resolved += TimedTextSource_Resolved;

var timedTextSourceUri_Pt = new
Uri("http://contoso.com/MyClipTimedText_pt.srt");
var timedTextSource_Pt =
TimedTextSource.CreateFromUri(timedTextSourceUri_Pt);
timedTextSourceMap[timedTextSource_Pt] = timedTextSourceUri_Pt;
timedTextSource_Pt.Resolved += TimedTextSource_Resolved;

// Add the TimedTextSource to the MediaSource
_mediaSource.ExternalTimedTextSources.Add(timedTextSource_En);
_mediaSource.ExternalTimedTextSources.Add(timedTextSource_Pt);

_mediaPlaybackItem = new MediaPlayerItem(_mediaSource);
_mediaPlaybackItem.TimedMetadataTracksChanged +=
MediaPlayerItem_TimedMetadataTracksChanged;

_mediaPlayer = new MediaPlayer();
_mediaPlayer.Source = _mediaPlaybackItem;
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);

```

In the handler for the [TimedTextSource.Resolved](#) event, check the **Error** property of the [TimedTextSourceResolveResultEventArgs](#) passed into the handler to determine if an error occurred while trying to load the timed text data. If the item was resolved successfully, you can use this handler to update additional properties of the resolved track. This example adds a label for each track based on the URI previously stored in the **Dictionary**.

C#

```

private void TimedTextSource_Resolved(TimedTextSource sender,
TimedTextSourceResolveResultEventArgs args)
{
    var timedTextSourceUri = timedTextSourceMap[sender];

    if (!(args.Error is null))
    {
        // Show that there was an error in your UI
        ShowMessageToUser($"There was an error resolving track:
{timedTextSourceUri}");
        return;
    }

    // Add a label for each resolved track
    var timedTextSourceUriString = timedTextSourceUri.AbsoluteUri;

```

```

if (timedTextSourceUriString.Contains("_en"))
{
    args.Tracks[0].Label = "English";
}
else if (timedTextSourceUriString.Contains("_pt"))
{
    args.Tracks[0].Label = "Portuguese";
}
}

```

For a list of the timed text formats that are supported on Windows, see [Supported codecs](#).

Add additional metadata tracks

You can dynamically create custom metadata tracks in code and associate them with a media source. The tracks you create can contain subtitle or caption text, or they can contain your proprietary app data.

Create a new [TimedMetadataTrack](#) by calling the constructor and specifying an ID, the language identifier, and a value from the [TimedMetadataKind](#) enumeration. Register handlers for the [CueEntered](#) and [CueExited](#) events. These events are raised when the start time for a cue has been reached and when the duration for a cue has expired, respectively.

Create a new cue object, appropriate for the type of metadata track you created, and set the ID, start time, and duration for the track. This example creates a data track, so a set of [DataCue](#) objects are generated and a buffer containing app-specific data is provided for each cue. To register the new track, add it to the [ExternalTimedMetadataTracks](#) collection of the [MediaSource](#) object.

Starting with Windows 10, version 1703, the [DataCue.Properties](#) property exposes a [PropertySet](#) that you can use to store custom properties in key/data pairs that can be retrieved in the [CueEntered](#) and [CueExited](#) events.

C#

```

TimedMetadataTrack metadataTrack = new TimedMetadataTrack("ID_0", "en-us",
TimedMetadataKind.Data);
metadataTrack.Label = "Custom data track";
metadataTrack.CueEntered += MetadataTrack_DataCueEntered;
metadataTrack.CueExited += MetadataTrack_CueExited;

// Example cue data
string data = "Cue data";
byte[] bytes = new byte[data.Length * sizeof(char)];

```

```

System.Buffer.BlockCopy(data.ToCharArray(), 0, bytes, 0, bytes.Length);
Windows.Storage.Streams.IBuffer buffer = bytes.AsBuffer();

for (int i = 0; i < 10; i++)
{
    DataCue cue = new DataCue();
    cue.Id = "ID_" + i;
    cue.Data = buffer;
    cue.Properties["AdUrl"] = "http://contoso.com/ads/123";
    cue.StartTime = TimeSpan.FromSeconds(3 + i * 3);
    cue.Duration = TimeSpan.FromSeconds(2);

    metadataTrack.AddCue(cue);
}

_mediaSource.ExternalTimedMetadataTracks.Add(metadataTrack);

```

The **CueEntered** event is raised when a cue's start time has been reached as long as the associated track has a presentation mode of **ApplicationPresented**, **Hidden**, or **PlatformPresented**. Cue events are not raised for metadata tracks while the presentation mode for the track is **Disabled**. This example simply outputs the custom data associated with the cue to the debug window.

C#

```

private void MetadataTrack_DataCueEntered(TimedMetadataTrack sender,
MediaCueEventArgs args)
{
    DataCue cue = (DataCue)args.Cue;
    string data =
System.Text.Encoding.Unicode.GetString(cue.Data.ToArray());
    System.Diagnostics.Debug.WriteLine("Cue entered: " + data);
    System.Diagnostics.Debug.WriteLine("Custom prop value: " +
cue.Properties["AdUrl"]);
}

```

This example adds a custom text track by specifying **TimedMetadataKind.Caption** when creating the track and using **TimedTextCue** objects to add cues to the track.

C#

```

TimedMetadataTrack metadataTrack = new TimedMetadataTrack("TrackID_0", "en-
us", TimedMetadataKind.Caption);
metadataTrack.Label = "Custom text track";
metadataTrack.CueEntered += MetadataTrack_TextCueEntered;

for (int i = 0; i < 10; i++)
{
    TimedTextCue cue = new TimedTextCue()
    {

```

```

        Id = "TextCueID_" + i,
        StartTime = TimeSpan.FromSeconds(i * 3),
        Duration = TimeSpan.FromSeconds(2)
    };

    cue.Lines.Add(new TimedTextLine() { Text = "This is a custom timed text cue." });
    metadataTrack.AddCue(cue);
}

_mediaSource.ExternalTimedMetadataTracks.Add(metadataTrack);

```

C#

```

private void MetadataTrack_TextCueEntered(TimedMetadataTrack sender,
MediaCueEventArgs args)
{
    TimedTextCue cue = (TimedTextCue)args.Cue;
    System.Diagnostics.Debug.WriteLine("Cue entered: " + cue.Id + " " +
cue.Lines[0].Text);
}

```

Play a list of media items with MediaPlayerPlaylist

The [MediaPlayerPlaylist](#) allows you to create a playlist of media items, which are represented by **MediaPlayerItem** objects.

Note Items in a [MediaPlayerPlaylist](#) are rendered using gapless playback. The system will use provided metadata in MP3 or AAC encoded files to determine the delay or padding compensation needed for gapless playback. If the MP3 or AAC encoded files don't provide this metadata, then the system determines the delay or padding heuristically. For lossless formats, such as PCM, FLAC, or ALAC, the system takes no action because these encoders don't introduce delay or padding.

To get started, declare a variable to store your **MediaPlayerPlaylist**.

C#

```

MediaPlayerPlaylist _mediaPlaybackList;

```

Create a **MediaPlayerItem** for each media item you want to add to your list using the same procedure described previously in this article. Initialize your **MediaPlayerPlaylist** object and add the media playback items to it. Register a handler for the [CurrentItemChanged](#) event. This event allows you to update your UI to reflect the

currently playing media item. You can also register for the [ItemOpened](#) event, which is raised when an item in the list is successfully opened, and the [ItemFailed](#) event, which is raised when an item in the list can't be opened.

Starting with Windows 10, version 1703, you can specify the maximum number of **MediaPlaybackItem** objects in the **MediaPlaybackList** that the system will keep open after they have been played by setting the [MaxPlayedItemsToKeepOpen](#) property. When a **MediaPlaybackItem** is kept open, playback of the item can start instantaneously when the user switches to that item because the item doesn't need to be reloaded. But keeping items open also increases the memory consumption of your app, so you should consider the balance between responsiveness and memory usage when setting this value.

To enable playback of your list, set the playback source of the **MediaPlayer** to your **MediaPlaybackList**.

C#

```
_mediaPlaybackList = new MediaPlaybackList();

var files = await filePicker.PickMultipleFilesAsync();

foreach (var file in files)
{
    var mediaPlaybackItem = new
MediaPlaybackItem(MediaSource.CreateFromStorageFile(file));
    _mediaPlaybackList.Items.Add(mediaPlaybackItem);
}

_mediaPlaybackList.CurrentItemChanged +=
MediaPlaybackList_CurrentItemChanged;
_mediaPlaybackList.ItemOpened += MediaPlaybackList_ItemOpened;
_mediaPlaybackList.ItemFailed += MediaPlaybackList_ItemFailed;

_mediaPlaybackList.MaxPlayedItemsToKeepOpen = 3;

_mediaPlayer = new MediaPlayer();
_mediaPlayer.Source = _mediaPlaybackList;
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
```

In the **CurrentItemChanged** event handler, update your UI to reflect the currently playing item, which can be retrieved using the [NewItem](#) property of the [CurrentMediaPlaybackItemChangedEventArgs](#) object passed into the event. Remember that if you update the UI from this event, you should do so within a call to [CoreDispatcher.RunAsync](#) so that the updates are made on the UI thread.

Starting with Windows 10, version 1703, you can check the [CurrentMediaPlaybackItemChangedEventArgs.Reason](#) property to get a value that indicates the reason that the item changed, such as the app switching items programmatically, the previously playing item reaching its end, or an error occurring.

C#

```
private void MediaPlayerList_CurrentItemChanged(MediaPlaybackList sender,
CurrentMediaPlaybackItemChangedEventArgs args) =>
    LogTelemetryData($"CurrentItemChanged reason:
{args.Reason.ToString()}");
```

Call [MovePrevious](#) or [MoveNext](#) to cause the media player to play the previous or next item in your **MediaPlaybackList**.

C#

```
private void prevButton_Click(object sender, RoutedEventArgs e) =>
_mediaPlaybackList.MovePrevious();
```

C#

```
private void nextButton_Click(object sender, RoutedEventArgs e) =>
_mediaPlaybackList.MoveNext();
```

Set the [ShuffleEnabled](#) property to specify whether the media player should play the items in your list in random order.

C#

```
private async void shuffleButton_Click(object sender, RoutedEventArgs e)
{
    _mediaPlaybackList.ShuffleEnabled = !_mediaPlaybackList.ShuffleEnabled;

    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        shuffleButton.FontWeight =
            _mediaPlaybackList.ShuffleEnabled ?
            Windows.UI.Text.FontWeights.Bold : Windows.UI.Text.FontWeights.Light;
    });
}
```

Set the [AutoRepeatEnabled](#) property to specify whether the media player should loop playback of your list.

C#

```
private async void autoRepeatButton_Click(object sender, RoutedEventArgs e)
{
    _mediaPlaybackList.AutoRepeatEnabled =
    !_mediaPlaybackList.AutoRepeatEnabled;

    await Dispatcher.RunAsync(Windows.UI.Core.CoreDispatcherPriority.Normal,
    () =>
    {
        autoRepeatButton.FontWeight =
            _mediaPlaybackList.AutoRepeatEnabled ?
            Windows.UI.Text.FontWeights.Bold : Windows.UI.Text.FontWeights.Light;
    });
}
```

Handle the failure of media items in a playback list

The [ItemFailed](#) event is raised when an item in the list fails to open. The [ErrorCode](#) property of the [MediaPlaybackItemError](#) object passed into the handler enumerates the specific cause of the failure when possible, including network errors, decoding errors, or encryption errors.

C#

```
private void MediaPlaybackList_ItemFailed(MediaPlaybackList sender,
MediaPlaybackItemFailedEventArgs args)
{
    LogError(args.Error.ErrorCode.ToString());
    LogError(args.Error.ExtendedError.HResult);
}
```

Disable playback of items in a playback list

Starting with Windows 10, version 1703, you can disable playback of one or more items in a [MediaPlaybackItemList](#) by setting the [IsDisabledInPlaybackList](#) property of a [MediaPlaybackItem](#) to false.

A typical scenario for this feature is for apps that play music streamed from the internet. The app can listen for changes in the network connection status of the device and disable playback of items that are not fully downloaded. In the following example, a handler is registered for the [NetworkInformation.NetworkStatusChanged](#) event.

C#

```
Windows.Networking.Connectivity.NetworkInformation.NetworkStatusChanged +=  
NetworkInformation_NetworkStatusChanged;
```

In the handler for **NetworkStatusChanged**, check to see if [GetInternetConnectionProfile](#) returns null, which indicates that the network is not connected. If this is the case, loop through all of the items in the playback list, and if the [TotalDownloadProgress](#) for the item is less than 1, meaning that the item has not fully downloaded, disable the item. If the network connection is enabled, loop through all of the items in the playback list and enable each item.

C#

```
private void NetworkInformation_NetworkStatusChanged(object sender)  
{  
    if  
(Windows.Networking.Connectivity.NetworkInformation.GetInternetConnectionPro  
file() == null)  
    {  
        // Check download status of each item in the list.  
(TotalDownloadProgress < 1 means not completely downloaded)  
        foreach (var item in _mediaPlaybackList.Items)  
        {  
            if (item.TotalDownloadProgress < 1)  
            {  
                item.IsDisabledInPlaybackList = true;  
            }  
        }  
    }  
    else  
    {  
        // Connected to internet, re-enable all playlist items  
        foreach (var item in _mediaPlaybackList.Items)  
        {  
            item.IsDisabledInPlaybackList = true;  
        }  
    }  
}
```

Defer binding of media content for items in a playback list by using MediaBinder

In the previous examples, a **MediaSource** is created from a file, URL, or stream, after which a **MediaPlaybackItem** is created and added to a **MediaPlaybackList**. For some scenarios, such as if the user is being charged for viewing content, you may want to defer the retrieval of the content of a **MediaSource** until the item in the playback list is ready to actually be played. To implement this scenario, create an instance of the

MediaBinder class. Set the **Token** property to an app-defined string that identifies the content for which you want to defer retrieval and then register a handler for the **Binding** event. Next, create a **MediaSource** from the **Binder** by calling **MediaSource.CreateFromMediaBinder**. Then, create a **MediaPlaybackItem** from the **MediaSource** and add it to the playback list as usual.

C#

```
_mediaPlaybackList = new MediaPlaybackList();

var binder = new MediaBinder();
binder.Token = "MyBindingToken1";
binder.Binding += Binder_Binding;
_mediaPlaybackList.Items.Add(new
MediaPlaybackItem(MediaSource.CreateFromMediaBinder(binder)));

binder = new MediaBinder();
binder.Token = "MyBindingToken2";
binder.Binding += Binder_Binding;
_mediaPlaybackList.Items.Add(new
MediaPlaybackItem(MediaSource.CreateFromMediaBinder(binder)));

_mediaPlayer = new MediaPlayer();
_mediaPlayer.Source = _mediaPlaybackList;
mediaPlayerElement.SetMediaPlayer(_mediaPlayer);
```

When the system determines that the content associated with the **MediaBinder** needs to be retrieved, it will raise the **Binding** event. In the handler for this event, you can retrieve the **MediaBinder** instance from the **MediaBindingEventArgs** passed into the event. Retrieve the string you specified for the **Token** property and use it to determine what content should be retrieved. The **MediaBindingEventArgs** provides methods for setting the bound content in several different representations, including **SetStorageFile**, **SetStream**, **SetStreamReference**, and **SetUri**.

C#

```
private void Binder_Binding(MediaBinder sender, MediaBindingEventArgs args)
{
    // Get a deferral if you need to perform async operations
    // var deferral = args.GetDeferral();

    var contentUri = new Uri("http://contoso.com/media/" +
args.MediaBinder.Token);
    args.SetUri(contentUri);

    // Call complete after your async operations are complete
    // deferral.Complete();
}
```

Note that if you are performing asynchronous operations, such as web requests, in the **Binding** event handler, you should call the [MediaBindingEventArgs.GetDeferral](#) method to instruct the system to wait for your operation to complete before continuing. Call [Deferral.Complete](#) after your operation is complete to instruct the system to continue.

Starting with Windows 10, version 1703, you can supply an [AdaptiveMediaSource](#) as bound content by calling [SetAdaptiveMediaSource](#). For more information on using adaptive streaming in your app, see [Adaptive streaming](#).

Related topics

- [Media playback](#)
- [Play audio and video with MediaPlayer](#)
- [Integrate with the System Media Transport Controls](#)
- [Play media in the background](#)

Feedback

Was this page helpful?

 Yes

 No

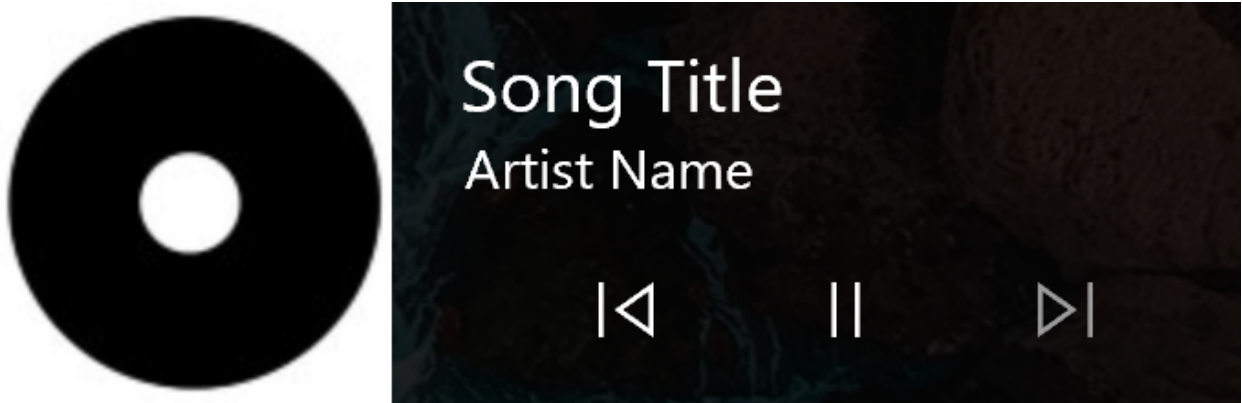
[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Integrate with the System Media Transport Controls

Article • 10/20/2022

This article shows you how to interact with the System Media Transport Controls (SMTC). The SMTC is a set of controls that are common to all Windows 10 devices and that provide a consistent way for users to control media playback for all running apps that use [MediaPlayer](#) for playback.

The System Media Transport Controls enable media application developers integrate with the built-in system UI to display media metadata such as artist, album title, or chapter title. The system transport control also allows a user to control the playback of a media app using the built-in system UI, such as pausing playback and skipping forward and backward in a playlist.



For a complete sample that demonstrates integration with the SMTC, see [System Media Transport Controls sample on github](#).

Automatic integration with SMTC

Starting with Windows 10, version 1607, UWP apps that use the [MediaPlayer](#) class to play media are automatically integrated with the SMTC by default. Simply instantiate a new instance of **MediaPlayer** and assign a [MediaSource](#), [MediaPlaybackItem](#), or [MediaPlaybackList](#) to the player's [Source](#) property and the user will see your app name in the SMTC and can play, pause, and move through your playback lists by using the SMTC controls.

Your app can create and use multiple **MediaPlayer** objects at once. For each active **MediaPlayer** instance in your app, a separate tab is created in the SMTC, allowing the user to switch between your active media players and those of other running apps. Whichever media player is currently selected in the SMTC is the one that the controls will affect.

For more information on using **MediaPlayer** in your app, including binding it to a **MediaPlayerElement** in your XAML page, see [Play audio and video with MediaPlayer](#).

For more information on working with **MediaSource**, **MediaPlaybackItem**, and **MediaPlaybackList**, see [Media items, playlists, and tracks](#).

Add metadata to be displayed by the SMTC

If you want add or modify the metadata that is displayed for your media items in the SMTC, such as a video or song title, you need to update the display properties for the **MediaPlaybackItem** representing your media item. First, get a reference to the **MediaItemDisplayProperties** object by calling [GetDisplayProperties](#). Next, set the type of media, music or video, for the item with the **Type** property. Then you can populate the fields of the **MusicProperties** or **VideoProperties**, depending on which media type you specified. Finally, update the metadata for the media item by calling [ApplyDisplayProperties](#).

C#

```
MediaItemDisplayProperties props = mediaPlaybackItem.GetDisplayProperties();  
props.Type = Windows.Media.MediaPlaybackType.Video;  
props.VideoProperties.Title = "Video title";  
props.VideoProperties.Subtitle = "Video subtitle";  
props.VideoProperties.Genres.Add("Documentary");  
mediaPlaybackItem.ApplyDisplayProperties(props);
```

C#

```
props = mediaPlaybackItem.GetDisplayProperties();  
props.Type = Windows.Media.MediaPlaybackType.Music;  
props.MusicProperties.Title = "Song title";  
props.MusicProperties.Artist = "Song artist";  
props.MusicProperties.Genres.Add("Polka");  
mediaPlaybackItem.ApplyDisplayProperties(props);
```

ⓘ Note

Apps should set a value for the **Type** property even if they aren't supplying other media metadata to be displayed by the System Media Transport Controls. This value helps the system handle your media content correctly, including preventing the screen saver from activating during playback.

Use CommandManager to modify or override the default SMTC commands

Your app can modify or completely override the behavior of the SMTC controls with the [MediaPlaybackCommandManager](#) class. A command manager instance can be obtained for each instance of the **MediaPlayer** class by accessing the [CommandManager](#) property.

For every command, such as the *Next* command which by default skips to the next item in a **MediaPlaybackList**, the command manager exposes a received event, like [NextReceived](#), and an object that manages the behavior of the command, like [NextBehavior](#).

The following example registers a handler for the **NextReceived** event and for the [IsEnabledChanged](#) event of the **NextBehavior**.

C#

```
_mediaPlayer.CommandManager.NextReceived += CommandManager_NextReceived;  
_mediaPlayer.CommandManager.NextBehavior.IsEnabledChanged +=  
NextBehavior_IsEnabledChanged;
```

The following example illustrates a scenario where the app wants to disable the *Next* command after the user has clicked through five items in the playlist, perhaps requiring some user interaction before continuing playing content. Each time the **NextReceived** event is raised, a counter is incremented. Once the counter reaches the target number, the [EnablingRule](#) for the *Next* command is set to [Never](#), which disables the command.

C#

```
int _nextPressCount = 0;  
private void CommandManager_NextReceived(MediaPlaybackCommandManager sender,  
MediaPlaybackCommandManagerNextReceivedEventArgs args)  
{  
    _nextPressCount++;  
    if (_nextPressCount > 5)  
    {  
        sender.NextBehavior.EnablingRule = MediaCommandEnablingRule.Never;  
        // Perform app tasks while the Next button is disabled  
    }  
}
```

You can also set the command to **Always**, which means the command will always be enabled even if, for the *Next* command example, there are no more items in the playlist.

Or you can set the command to **Auto**, where the system determines whether the command should be enabled based on the current content being played.

For the scenario described above, at some point the app will want to reenable the *Next* command and does so by setting the **EnablingRule** to **Auto**.

C#

```
_mediaPlayer.CommandManager.NextBehavior.EnablingRule =  
MediaCommandEnablingRule.Auto;  
_nextPressCount = 0;
```

Because your app may have it's own UI for controlling playback while it is in the foreground, you can use the **IsEnabledChanged** events to update your own UI to match the SMTC as commands are enabled or disabled by accessing the **IsEnabled** of the **MediaPlaybackCommandManagerCommandBehavior** passed into the handler.

C#

```
private void  
NextBehavior_IsEnabledChanged(MediaPlaybackCommandManagerCommandBehavior  
sender, object args)  
{  
    MyNextButton.IsEnabled = sender.IsEnabled;  
}
```

In some cases, you may want to completely override the behavior of an SMTC command. The example below illustrates a scenario where an app uses the *Next* and *Previous* commands to switch between internet radio stations instead of skipping between tracks in the current playlist. As in the previous example, a handler is registered for when a command is received, in this case it is the **PreviousReceived** event.

C#

```
_mediaPlayer.CommandManager.PreviousReceived +=  
CommandManager_PreviousReceived;
```

In the **PreviousReceived** handler, first a **Deferral** is obtained by calling the **GetDeferral** of the **MediaPlaybackCommandManagerPreviousReceivedEventArgs** passed into the handler. This tells the system to wait for until the deferral is complete before executing the command. This is extremely important if you are going to make asynchronous calls in the handler. At this point, the example calls a custom method that returns a **MediaPlaybackItem** representing the previous radio station.

Next, the **Handled** property is checked to make sure that the event wasn't already handled by another handler. If not, the **Handled** property is set to true. This lets the SMTC, and any other subscribed handlers, know that they should take no action to execute this command because it has already been handled. The code then sets the new source for the media player and starts the player.

Finally, **Complete** is called on the deferral object to let the system know that you are done processing the command.

C#

```
private async void
CommandManager_PreviousReceived(MediaPlaybackCommandManager sender,
MediaPlaybackCommandManagerPreviousReceivedEventArgs args)
{
    var deferral = args.GetDeferral();
    MediaPlaybackItem mediaPlaybackItem = await GetPreviousStation();

    if(args.Handled != true)
    {
        args.Handled = true;
        sender.MediaPlayer.Source = mediaPlaybackItem;
        sender.MediaPlayer.Play();
    }
    deferral.Complete();
}
```

Manual control of the SMTC

As mentioned previously in this article, the SMTC will automatically detect and display information for every instance of **MediaPlayer** that your app creates. If you want to use multiple instances of **MediaPlayer** but want the SMTC to provide a single entry for your app, then you must manually control the behavior of the SMTC instead of relying on automatic integration. Also, if you are using **MediaTimelineController** to control one or more media players, you must use manual SMTC integration. Also, if your app uses an API other than **MediaPlayer**, such as the **AudioGraph** class, to play media, you must implement manual SMTC integration for the user to use the SMTC to control your app. For information on how to manually control the SMTC, see [Manual control of the System Media Transport Controls](#).

Related topics

- [Media playback](#)
- [Play audio and video with MediaPlayer](#)

- [Manual control of the System Media Transport Controls](#)
- [System Media Transport Controls sample on github](#) 

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Manual control of the System Media Transport Controls

Article • 10/20/2022

Starting with Windows 10, version 1607, UWP apps that use the [MediaPlayer](#) class to play media are automatically integrated with the System Media Transport Controls (SMTC) by default. This is the recommended way of interacting with the SMTC for most scenarios. For more information on customizing the SMTC's default integration with [MediaPlayer](#), see [Integrate with the System Media Transport Controls](#).

There are a few scenarios where you may need to implement manual control of the SMTC. These include if you are using a [MediaTimelineController](#) to control playback of one or more media players. Or if you are using multiple media players and only want to have one instance of SMTC for your app. You must manually control the SMTC if you are using [MediaElement](#) to play media.

Set up transport controls

If you are using [MediaPlayer](#) to play media, you can get an instance of the [SystemMediaTransportControls](#) class by accessing the [MediaPlayer.SystemMediaTransportControls](#) property. If you are going to manually control the SMTC, you should disable the automatic integration provided by [MediaPlayer](#) by setting the [CommandManager.IsEnabled](#) property to false.

ⓘ Note

If you disable the [MediaPlaybackCommandManager](#) of the [MediaPlayer](#) by setting [IsEnabled](#) to false, it will break the link between the [MediaPlayer](#) the [TransportControls](#) provided by the [MediaPlayerElement](#), so the built-in transport controls will no longer automatically control the playback of the player. Instead, you must implement your own controls to control the [MediaPlayer](#).

C#

```
_mediaPlayer = new MediaPlayer();  
_systemMediaTransportControls = _mediaPlayer.SystemMediaTransportControls;  
_mediaPlayer.CommandManager.IsEnabled = false;
```